Department of Computer Science & Engineering
The University of Texas at Arlington

Detailed Design Specification
CSE 4317: Senior Design II
Fall 2024



AccessVR
VR Wheelchair Controller

Jake Cooper
Eric Nguyen
Khoa Nguyen
Santiago Hernandez

Revision History

Revision	Date	Author(s)	Description
0.1	9.21.2024	KN, EN, JC, SH	document creation

CONTENTS

1 Introduction **5**
- 1.1 Purpose and Use . . . 5
- 1.2 Key Requirements . . . 5

2 System Overview **5**

3 Headset Subsystems **6**
- 3.1 Layer Hardware . . . 6
- 3.2 Layer Operating System . . . 6
- 3.3 Layer Software Dependencies . . . 7
- 3.4 Player Movement Subsystem . . . 7
- 3.5 Hand Tracking Subsystem . . . 7
- 3.6 UI Subsystem . . . 8
- 3.7 Environment Subsystem . . . 9

4 Data Processing Layer Subsystems **11**
- 4.1 Layer Hardware . . . 11
- 4.2 Layer Operating System . . . 11
- 4.3 Layer Software Dependencies . . . 11
- 4.4 Microcontroller Subsystem . . . 11
- 4.5 API Subsystem . . . 13

5 Wheelchair Layer Subsystem **15**
- 5.1 Layer Hardware . . . 15
- 5.2 Layer Operating System . . . 15
- 5.3 Layer Software Dependencies . . . 15
- 5.4 Subsystem 1: Button Cluster . . . 15
- 5.5 Subsystem 2: Encoder . . . 16

6 Appendix A **17**

List of Figures

1 System architecture 6
2 Example subsystem description diagram 7
3 Example subsystem description diagram 8
4 Example subsystem description diagram 8
5 Example subsystem description diagram 9
6 Microcontroller subsystem diagram 11
7 API subsystem diagram 13
8 Wheelchair subsystem diagram 15
9 Wheelchair subsystem diagram 16

List of Tables

1 Introduction

Access VR is a product that the aims to streamline and aid physical therapy. It allows users to be able to practice various scenarios and movements in a safe and controlled virtual space, thereby improving their confidence and skills in real-world wheelchair navigation. This will hopefully expedite the recovery and learning process of using a wheelchair for newly wheelchair bound users. Additionally this technology can be used to allow for wheelchair bounds users to have more accessibility in virtual environments as it replicates wheelchair movement more accurately than traditionally controllers.

1.1 Purpose and Use

The VR wheelchair system aims to provide an innovative training platform for wheelchair users to practice and improve their navigation skills within a virtual reality environment. By simulating real-world scenarios and obstacles, the system allows users to experience and adapt to various challenges without physical risk. This tool is intended to be used in rehabilitation centers, training facilities, and by individuals at home who seek to enhance their wheelchair maneuvering abilities. The system integrates with a physical wheelchair to provide a realistic and responsive experience that mimics actual movement and interaction within virtual settings.

1.2 Key Requirements

- Virtual Environments: The system includes a variety of carefully designed VR environments that simulate different real-life scenarios. These environments provide users with opportunities to practice and enhance their navigation skills in a safe and controlled setting.
- User Interface: The VR system features an intuitive user interface that allows users to easily interact with the virtual environment. The interface is designed to be accessible and user-friendly, ensuring that wheelchair users can navigate the VR world with ease.
- Sensor-Based Modules: The system utilizes sensor-based modules that attach to the wheels of the wheelchair. These sensors capture the movement of the wheels and translate it into corresponding actions within the VR environment. This allows users to control their movement in the VR world by spinning the wheels in real life.
- Customization and Adaptability: The system can be customized to meet the specific needs of individual users. This includes adjusting the sensitivity of the sensors, the difficulty of the VR environments, and other settings to ensure an optimal training experience.

2 System Overview

The Headset layer serves as the user's interface with the virtual environment, enabling movement and interaction via hand tracking and player movement. This allows the user to navigate the virtual space and control the user interface seamlessly. The Data Processing layer acts as a bridge between the wheelchair and the headset. It processes inputs from the wheelchair through an Arduino microcontroller, which sends the data to an API. This API translates the wheelchair movements into data that the headset can use. The Wheelchair layer encompasses the hardware setup, capturing the user's movements and transmitting them to the data processing system for integration into the virtual experience.

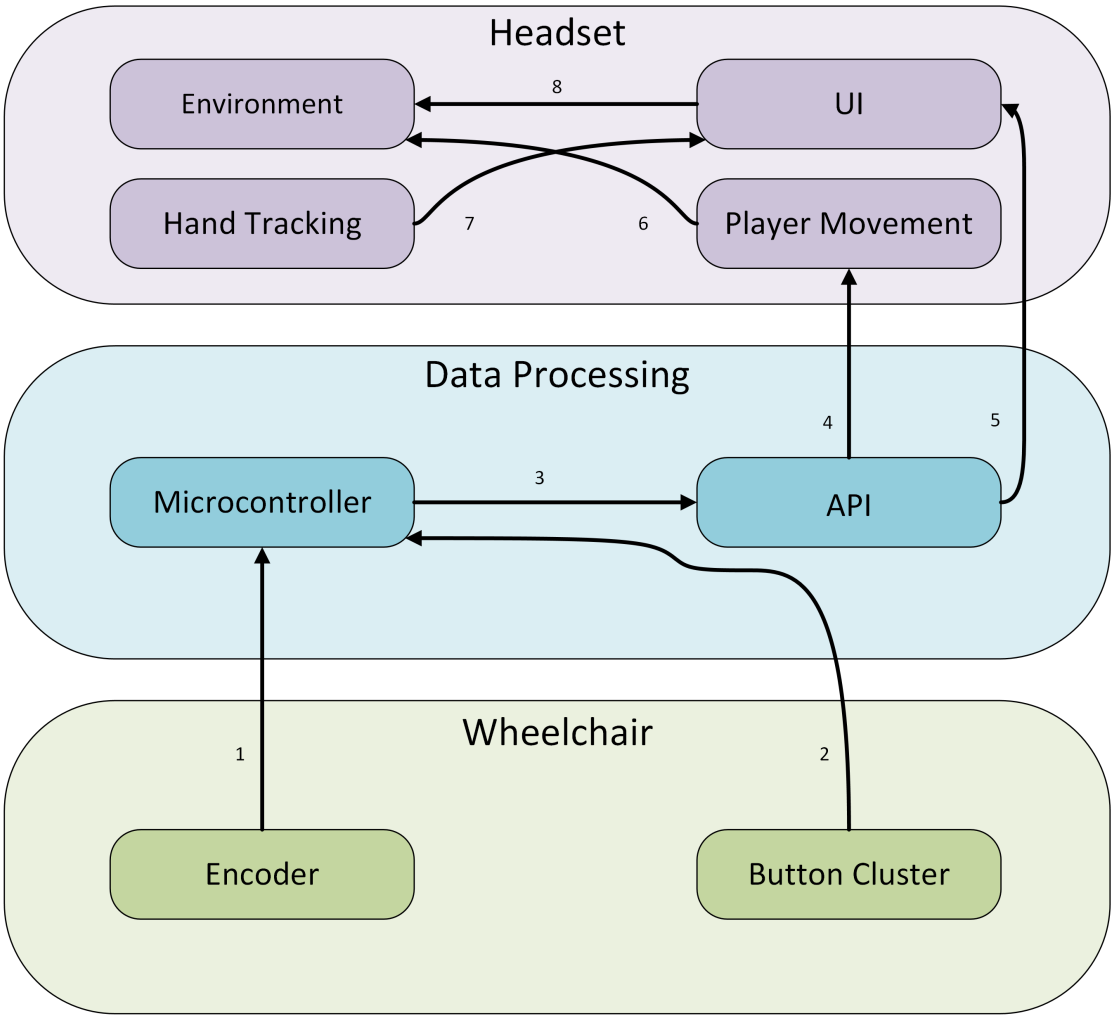


Figure 1: System architecture

3 Headset Subsystems

This layer comprises of four subsystems: The Environment, UI, Hand Tracking, and player Movement. Its primary responsibilities include Moving the player through the environment and interacting with it through Hand tracking and the users UI.

3.1 Layer Hardware

All Headset subsystems will be implemented as software processes running on a Meta Quest 3 through a PC. The Meta Quest 3 serves as the core hardware component for the layer, responsible for managing the operations of its subsystems. It features necessary processing power and I/O capabilities to handle an unreal engine environment.

3.2 Layer Operating System

This layer requires a computer with an operating system compatible with PCVR gaming and Unreal engine. In this case, the system runs on Windows 10/11, which supports the necessary software tools for programming and interfacing with the meta quest 3.

3.3 Layer Software Dependencies

This project uses the Unreal Engine framework and the MetaXR SDK.

3.4 Player Movement Subsystem

The player movement subsystem is a software component responsible for moving the player. It gathers uses data gathered by the Data Processing Layer and uses it to move the player across a virtual environment.

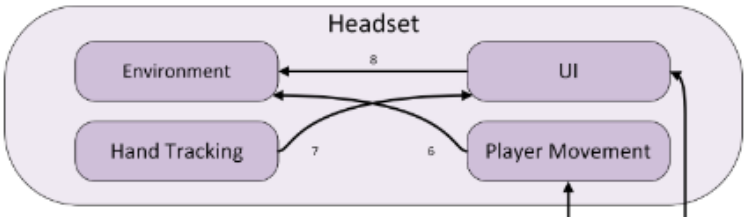


Figure 2: Example subsystem description diagram

3.4.1 Subsystem Hardware

The Player movement Subsystem uses the Meta Quest 3 headset to know the players location.

3.4.2 Subsystem Operating System

The player movement subsystem does not require a specific operating system. In this case, the system runs on Windows 10/11 on PC and the Meta Horizon OS, which supports the necessary software tools for programming and interfacing with the meta quest 3.

3.4.3 Subsystem Software Dependencies

Player movement is done through the Unreal Engine framework.

3.4.4 Subsystem Programming Languages

C++ was used in the making of AccessVR, along with Unreal Engines Blueprints System.

3.4.5 Subsystem Data Structures

FVectors and FQuads will be used to track the players position and rotation in real-time.

3.4.6 Subsystem Data Processing

Through the utilization of blueprints. Data will be processed so that the player will move either left or right depending on the rotation of the wheel.

3.5 Hand Tracking Subsystem

The Hand Tracking subsystem is the software component responsible for allowing the player to interact with the virtual world without the need for controllers. Gathers user hand data from the quest 3 cameras to be used to navigate the UI.

3.5.1 Subsystem Hardware

The subsystem uses two outer hybrid monochrome visible light/infrared cameras on the quest 3 to capture the position of the users hands.

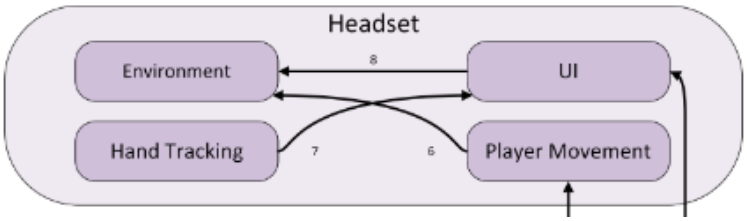


Figure 3: Example subsystem description diagram

3.5.2 Subsystem Operating System

The Hand Tracking subsystem requires the Meta Horizon OS, which supports the necessary software tools for programming and interfacing with the meta quest 3. For PCVR in Unreal the system runs on Windows 10/11 on PC.

3.5.3 Subsystem Software Dependencies

Hand tracking is done through the Unreal Engine Meta XR Plugin.

3.5.4 Subsystem Programming Languages

The Hand-Tracking subsystem is built using Unreal Engines Blueprints and C++.

3.5.5 Subsystem Data Structures

FTransform is used for the users joint orientation and position. To manage different hand gestures the team will look into State machine States.

3.5.6 Subsystem Data Processing

Through the utilization of blueprints and C++, Wheelchair Locomotion with Physics-Based movement will be implemented to allow for hand push input and directional control.

3.6 UI Subsystem

The UI subsystem is the software component responsible for gathering data from the data processing layer and hand-Tracking to interact with the virtual environment. The UI will give the user options on how to navigate AccessVR and its levels.

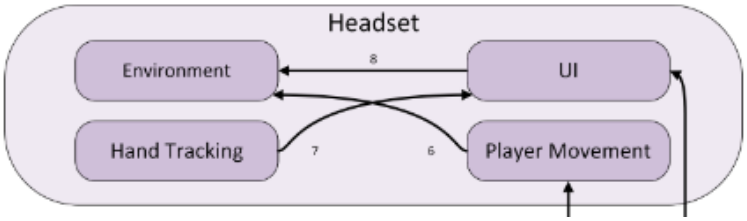


Figure 4: Example subsystem description diagram

3.6.1 Subsystem Hardware

The UI subsystem uses the Meta Quest 3 to display the UI to the user.

3.6.2 Subsystem Operating System

The UI subsystem does not require a specific operating system. In this case, the system runs on Windows 10/11 on PC and the Meta Horizon OS, which supports the necessary software tools for programming and interfacing with the meta quest 3.

3.6.3 Subsystem Software Dependencies

UI is handled through The Unreal Engine Framework using Figma based designs.

3.6.4 Subsystem Programming Languages

The UI subsystem uses the C++ Programming language and UE's blueprints.

3.6.5 Subsystem Data Structures

The UI subsystem uses UWidget Components for 3D widgets. These will be used as floating UI components in VR.

3.6.6 Subsystem Data Processing

Gesture recognition will be used to give meaning to user hand gestures in Vr so that the UI is interactable.

3.7 Environment Subsystem

The Environment subsystem is the software component responsible for providing the user with paths to navigate using their wheelchair player controller.Its primary function is to ensure that the player can seamlessly navigate the virtual environment, particularly in areas with potential obstacles such as stairs or ramps.

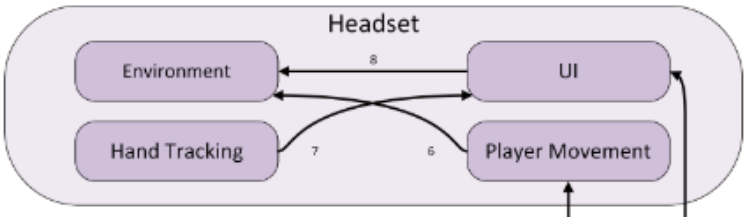


Figure 5: Example subsystem description diagram

3.7.1 Subsystem Hardware

The Meta Quest 3 is responsible for showing the world to the user through their PC via Airlink.

3.7.2 Subsystem Operating System

The Environment subsystem requires the Meta Horizon OS, which supports the necessary software tools for programming and interfacing with the meta quest 3. For PCVR in Unreal the system runs on Windows 10/11 on PC.

3.7.3 Subsystem Software Dependencies

The Environment is done through the Unreal Engine Framework using Blender built assets.

3.7.4 Subsystem Programming Languages

The programming for the Environment subsystem was done in C++/Blueprints and the assets were created in Blender.

3.7.5 Subsystem Data Structures

The GameMode class will be used for the environments game.

3.7.6 Subsystem Data Processing

The coin game algorithm would complete the game once the user collects all the coins in the map. Used to show the player the chosen paths.

4 Data Processing Layer Subsystems

This layer comprises two subsystems: the microcontroller and the API. Its primary responsibilities include gathering raw data, converting it into meaningful input, and transmitting it to the Headset Layer.

4.1 Layer Hardware

Both subsystems, the microcontroller and the API, will be implemented as software processes running on an embedded computer. This embedded computer serves as the core hardware component for the layer, responsible for managing the operations of both subsystems. It features necessary processing power and I/O capabilities to handle data collection, processing, and transmission tasks.

4.2 Layer Operating System

This layer requires a computer with an operating system compatible with the Arduino development environment. In this case, the system runs on Windows 11, which supports the necessary software tools for programming and interfacing with the microcontroller. Windows 11 ensures smooth integration with the Arduino IDE, allowing for efficient development, data processing, and communication between the subsystems and other components.

4.3 Layer Software Dependencies

There are no specific software dependencies at the layer level, as all necessary libraries and frameworks are handled within the individual subsystems.

4.4 Microcontroller Subsystem

The microcontroller subsystem is a hardware component responsible for gathering sensor data and processing it into meaningful input. It serves as the interface between the physical environment and the software system, collecting raw data from rotary encoders. This data is then translated into a format that can be transmitted to the next subsystem, API, ensuring seamless integration with the overall system. The microcontroller operates independently, executing its tasks in real time and communicating with the rest of the system through serial connections.

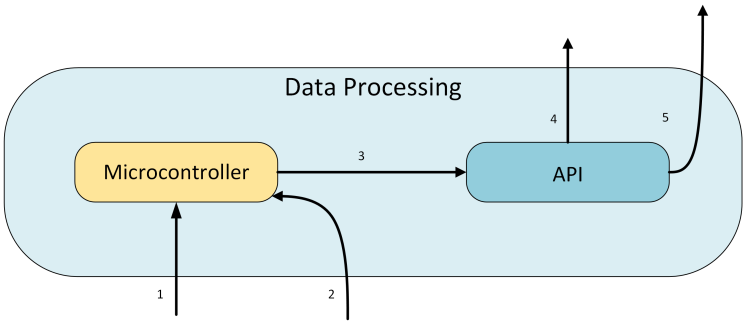


Figure 6: Microcontroller subsystem diagram

4.4.1 Subsystem Hardware

The microcontroller subsystem uses the Elegoo Uno R3, which is an Arduino-compatible microcontroller board based on the ATmega328P. This hardware component includes 14 digital input/output pins, 6 analog inputs, a 16 MHz quartz crystal, and a USB connection. It is responsible for interfacing with sensors and other hardware devices to collect raw data, process it into meaningful inputs, and transmit it to the connected computer. The rotary encoder is connected via 5V, GND, digital 2 and 3 pins. The Elegoo Uno R3 is powered via a USB connection to the computer.

4.4.2 Subsystem Operating System

The microcontroller subsystem does not require a specific operating system, as the Elegoo Uno R3 operates independently of an OS. It is programmed directly using the Arduino IDE on the host computer, which manages the development and uploading of code to the microcontroller. Therefore, there are no additional operating system requirements for this subsystem.

4.4.3 Subsystem Software Dependencies

This subsystem requires one software dependency to function properly:

- Arduino IDE and Serial Communication Libraries: The Arduino IDE is essential for programming and uploading code to the microcontroller. The layer relies on Arduino's built-in libraries like Serial for communication between the microcontroller and the computer.

4.4.4 Subsystem Programming Languages

The microcontroller subsystem primarily uses Arduino as the programming language, specifically through the Arduino programming environment. The programming language is a simplified version of C++ programming language.

4.4.5 Subsystem Data Structures

For the microcontroller subsystem, data being transmitted from the Elegoo Uno R3 to the PC is assembled into packets for efficient communication. Each packet consists of the following structure:

- Header: Contains metadata such as the packet type (e.g., sensor data, status information) and a sequence number for tracking.
- Payload: This is the main data section, which includes the encoder readings or meaningful input values collected by the microcontroller. The payload may vary in size depending on the number of sensors and the type of data being transmitted.
- Checksum: A simple error-checking mechanism that ensures data integrity during transmission. The checksum is calculated based on the contents of the packet and is used to verify that the data received matches what was sent.

4.4.6 Subsystem Data Processing

The microcontroller subsystem employs several algorithms and processing strategies to effectively manage data from rotary encoders. Key algorithms include:

- RPM Calculation: To extract the revolutions per minute (RPM) from the rotary encoder, the microcontroller measures the time interval between successive pulses generated by the encoder. By calculating the number of pulses in a given time frame and converting this to RPM, the system can determine the speed of rotation.
- Direction Determination: To determine the direction of movement (forward or backward), the microcontroller analyzes the sequence of pulses from the rotary encoder. If the pulses are received in a specific order, it indicates a forward motion, while the reverse order signifies backward motion.
- Differential Movement Calculation: To assess lateral movement (left or right), the microcontroller compares the readings from two rotary encoders. By calculating the difference in RPM between the two encoders, it can determine the turning direction and the degree of rotation. This allows for precise control of the system's navigation and maneuvering.

4.5 API Subsystem

The API subsystem serves as a software interface that facilitates communication between the microcontroller subsystem and the Headset Layer in the VR wheelchair system. This subsystem is implemented as a set of classes and functions designed to handle data transmission, processing, and interaction with Unreal Engine.

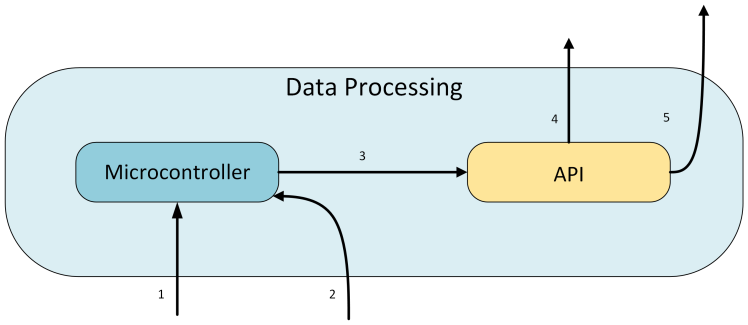


Figure 7: API subsystem diagram

4.5.1 Subsystem Hardware

The API subsystem does not require any specific hardware components, as it operates entirely as a software interface. It relies on the existing hardware of the microcontroller and the computer to facilitate communication and data processing.

4.5.2 Subsystem Operating System

The API subsystem does not have specific operating system requirements, as it functions entirely within the context of the host computer's operating system. Since the subsystem communicates with the microcontroller and interfaces with Unreal Engine, it operates on the existing environment provided by Windows 11.

4.5.3 Subsystem Software Dependencies

This subsystem requires these software dependencies to function properly:

- Unreal Engine: Unreal Engine is the main framework used to develop the VR experience. It provides the tools necessary to map data from the microcontroller to the virtual environment, using its powerful Blueprint visual scripting system.
- Unreal Engine Serial Communication Plugin: To interface with the Arduino, you may need a serial communication plugin for Unreal Engine. This allows the data received from the microcontroller via the computer to be processed and integrated into the VR environment.

4.5.4 Subsystem Programming Languages

The API subsystem primarily utilizes C++ as the programming language for developing the underlying code that interfaces with Unreal Engine. C++ is used to create classes and functions that manage data transmission between the microcontroller and the VR application. Additionally, the Blueprint scripting language within Unreal Engine is employed to visually map the data received from the microcontroller to the virtual environment.

4.5.5 Subsystem Data Structures

The API subsystem primarily relies on the built-in SerialCommunicator class for managing data transmission. Since this class handles the essential functions for communication between the microcontroller and Unreal Engine, there are no additional custom classes or data structures worth discussing at this time.

4.5.6 Subsystem Data Processing

For the API subsystem, there aren't specific algorithms or processing strategies beyond the built-in functionalities of the SerialCommunicator class and the mapping done with Blueprints in Unreal Engine. The core tasks involve:

- Data Reception: The SerialCommunicator efficiently receives data packets from the microcontroller, leveraging its built-in methods for handling serial communication.
- Data Mapping: The process of mapping the received data to the virtual environment in Unreal Engine is primarily visual and handled through the Blueprint system. This involves connecting input nodes to variables and functions that represent the state of the VR environment.

5 Wheelchair Layer Subsystem

5.1 Layer Hardware

This layer is our main physical and hardware layer. The wheelchair is suspended in the air via stands so that the wheels can move in place. Attached to each wheel is a rotary encoder to track the speed and direction that the wheelchair wheels are moving. In addition, each arm of the wheelchair will have a single button to aid the user in navigating the simulation.

5.2 Layer Operating System

No specific Operating System

5.3 Layer Software Dependencies

No specific Software Dependencies System

5.4 Subsystem 1: Button Cluster

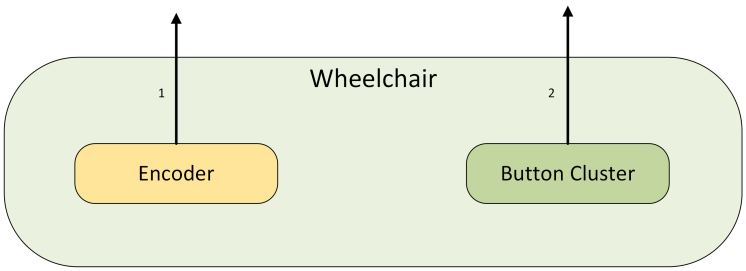


Figure 8: Wheelchair subsystem diagram

5.4.1 Subsystem Hardware

A single button attached to each arm of the wheelchair will provide an additional method of interaction with the headset.

5.4.2 Subsystem Operating System

No specific operating systems for this layer

5.4.3 Subsystem Software Dependencies

Only software dependencies in this layer are included in the Arduino development environment.

5.4.4 Subsystem Programming Languages

C Like Arduino programming language.

5.4.5 Subsystem Data Structures

The Data is a simple on/off signal generated by the button and read by the microprocessor.

5.4.6 Subsystem Data Processing

After a button is pressed, the microprocessor will read the signal generated by the button press and generate an interrupt to be sent to the next layer of data processing.

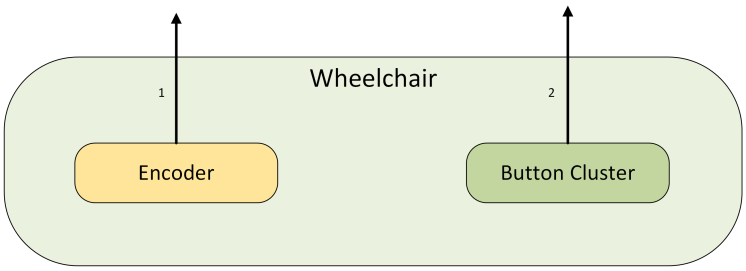


Figure 9: Wheelchair subsystem diagram

5.5 Subsystem 2: Encoder

5.5.1 Subsystem Hardware

The wheelchair will have a rotary encoder attached to each wheel. These encoders will serve to track wheel movement, which can later be translated into player movement in the VR environment.

5.5.2 Subsystem Operating System

No specific operating systems for this layer

5.5.3 Subsystem Software Dependencies

Only software dependencies in this layer are included in the Arduino development environment.

5.5.4 Subsystem Programming Languages

C- like Arduino programming language.

5.5.5 Subsystem Data Structures

The Data is relatively simple and basic integers are passed from the micro controller to the data processing layer. This can seem analog-like, but mainly just passes the current position of the wheel rotation to the next layer.

5.5.6 Subsystem Data Processing

Most of the Data processing will be be done in the data processing layer. However, the encoder is designed to have a counter and work with absolute position. It counts up in one direction and down in the other. It is meant to be very fine and precise so that the movement follows.

6 APPENDIX A

All files can be found on the shared Github repository:
https://github.com/Vidssk/AccessVR.githttps://github.com/Vidssk/AccessVR.git

References